UNIVERSIDAD NACIONAL DE SAN ANTONIO ABAD DEL CUSCO

CARRERA PROFESIONAL DE INGENIERÍA INFORMÁTICA Y DE SISTEMAS

GUÍA DE LABORATORIO N.º 05

Splay Trees y árboles dinámicos

Asignatura:	Algoritmos Avanzados
Unidad:	II. Estructuras de datos avanzadas
Semestre académico:	2026-II
Duración:	120 minutos
Organización:	Trabajo grupal en los equipos del proyecto semestral
Integrantes:	Aproximadamente cuatro estudiantes por equipo
Lenguaje:	Python 3
Docente:	Héctor Eduardo Ugarte Rojas

Equipo: _____

Integrantes: _____

Fecha: _____

Índice

1. Objetivo de la práctica **2**

2. Trabajo preparatorio obligatorio **2**

3. Materiales **2**

4. Marco teórico breve **3**

4.1. Splay Trees . . . 3

4.2. Árbol representado y árboles auxiliares . . . 3

4.3. Operación `EXPOSE` . . . 3

4.4. Operaciones sobre el bosque . . . 3

5. Especificación mínima de la implementación **4**

5.1. Clase del Splay Tree . . . 4

5.2. Clase del Link-Cut Tree . . . 4

6. Invariantes que deben comprobarse **5**

7. Distribución sugerida de los 120 minutos **5**

8. Ejercicios resueltos **5**

8.1. Ejercicio resuelto 1: ejecutar e instrumentar `splay` . . . 5

8.2. Ejercicio resuelto 2: búsqueda, inserción y eliminación . . . 7

8.3. Ejercicio resuelto 3: modelo de referencia para el bosque dinámico . . . 9

9. Ejercicios propuestos **10**

9.1. Ejercicio propuesto 1: implementar y validar el Link-Cut Tree . . . 10

9.2. Ejercicio propuesto 2: pruebas diferenciales y análisis experimental . . . 11

10. Resultados mínimos requeridos **12**

11. Preguntas de análisis **13**

12. Producto de entrega **13**

13. Tabla de evaluación **14**

1. Objetivo de la práctica

Objetivo

Implementar y validar las operaciones fundamentales de un Splay Tree y utilizar esa estructura autoajustable como base de los árboles dinámicos Link-Cut. Posteriormente, implementar un bosque dinámico con operaciones de exposición de caminos, cambio de raíz, conectividad, enlace, corte y consulta agregada sobre caminos.

Al finalizar la práctica, el equipo será capaz de:

1. implementar rotaciones y los casos *zig*, *zig-zig* y *zig-zag*;
2. implementar metodo splay, búsqueda, inserción, eliminación, `split` y `join` en un Splay Tree;
3. comprobar que las rotaciones conservan el recorrido *in-order*;
4. distinguir el árbol representado de los Splay Trees auxiliares;
5. implementar `EXPOSE`, `MAKE-ROOT`, `FIND-ROOT`, `LINK` y `CUT`;
6. mantener información agregada para responder una consulta sobre caminos;
7. comprobar invariantes mediante pruebas deterministas y aleatorias;
8. interpretar la garantía de $\mathcal{O}(\log n)$ amortizado por operación.

2. Trabajo preparatorio obligatorio

Cada integrante deberá revisar:

1. las rotaciones izquierda y derecha de un árbol binario de búsqueda;
2. los casos *zig*, *zig-zig* y *zig-zag*;
3. búsqueda exitosa y fallida en un Splay Tree;
4. eliminación mediante separación de subárboles y `join`;
5. diferencia entre árbol representado y Splay Tree auxiliar;
6. caminos preferidos, aristas sólidas y aristas discontinuas;
7. significado de `EXPOSE`, `FIND-ROOT`, `LINK` y `CUT`;
8. costo real, peor caso y costo amortizado.

3. Materiales

- Computadora con Python 3.11 o posterior, recomendado.
- Editor o IDE, terminal y Git.
- Biblioteca estándar: `random`, `time`, `statistics`, `csv`, `collections` y `dataclasses`.
- Repositorio del equipo con la estructura solicitada.
- Diapositivas del tema 5 y casos de prueba preparados por el equipo.
- Herramienta para generar tablas y gráficos a partir del archivo CSV.

4. Marco teórico breve

4.1. Splay Trees

Un Splay Tree es un árbol binario de búsqueda autoajustable. Después de buscar, insertar o localizar un nodo para eliminarlo, se ejecuta `splay` sobre el nodo involucrado. Una operación individual puede costar $\Theta(n)$, pero una secuencia de operaciones tiene costo amortizado $\mathcal{O}(\log n)$ por operación.

Caso	Configuración	Rotaciones
Zig	El padre es la raíz	Una rotación
Zig-zig	Nodo y padre están en el mismo lado	Dos rotaciones en la misma dirección
Zig-zag	Nodo y padre están en lados opuestos	Dos rotaciones en direcciones opuestas

4.2. Árbol representado y árboles auxiliares

Un Link-Cut Tree mantiene un bosque de árboles enraizados. El árbol conceptual de la aplicación se denomina **árbol representado**. Los caminos preferidos del árbol representado se almacenan mediante Splay Trees auxiliares.

Las rotaciones cambian la forma de los árboles auxiliares, pero no cambian las aristas del bosque representado. En cada árbol auxiliar, el recorrido *in-order* conserva el orden de los vértices sobre el camino representado.

4.3. Operación `EXPOSE`

La operación `EXPOSE(v)` reorganiza la descomposición preferida para que el camino desde la raíz representada hasta v quede almacenado en un único Splay Tree auxiliar, con v en su raíz auxiliar. La operación cambia la representación interna, pero no la conectividad del bosque representado.

4.4. Operaciones sobre el bosque

Operación	Efecto	Costo amortizado
`FIND-ROOT(v)`	Encuentra la raíz representada de la componente de v.	$\mathcal{O}(\log n)$
`MAKE-ROOT(v)`	Convierte a v en raíz representada invirtiendo el camino expuesto.	$\mathcal{O}(\log n)$
`LINK(u,v)`	Une dos componentes diferentes mediante una arista.	$\mathcal{O}(\log n)$
`CUT(u,v)`	Elimina una arista existente y divide una componente.	$\mathcal{O}(\log n)$
`PATH-SUM (Adicional a las diapositivas)(u,v)`	Suma valores sobre el camino simple entre dos nodos conectados.	$\mathcal{O}(\log n)$

5. Especificación mínima de la implementación

5.1. Clase del Splay Tree

Listing 1: Interfaz mínima del Splay Tree

```python
class SplayNode:
    def __init__(self, key, value=None):
        self.key = key
        self.value = value
        self.left = None
        self.right = None
        self.parent = None


class SplayTree:
    def search(self, key): ...
    def insert(self, key, value=None): ...
    def delete(self, key): ...
    def split(self, key): ...
    def join(self, other): ...

    # Metodos internos
    def _rotate(self, node): ...
    def _splay(self, node): ...
```

5.2. Clase del Link-Cut Tree

Listing 2: Interfaz mínima del Link-Cut Tree

```python
class LinkCutNode:
    def __init__(self, vertex, value=0):
        self.vertex = vertex
        self.value = value
        self.left = None
        self.right = None
        self.parent = None
        self.reversed = False
        self.path_sum = value
        self.size = 1


class LinkCutForest:
    def expose(self, node): ...
    def make_root(self, node): ...
    def find_root(self, node): ...
    def connected(self, u, v): ...
    def link(self, u, v): ...
    def cut(self, u, v): ...
    def path_sum(self, u, v): ...

    # Metodos internos
    def _is_aux_root(self, node): ...
    def _push(self, node): ...
    def _update(self, node): ...
    def _rotate(self, node): ...
    def _splay(self, node): ...
```

Manejadores de nodos

La creación de un vértice debe devolver el objeto que lo representa. Las operaciones del Link-Cut Tree se ejecutarán sobre esos manejadores, sin buscar linealmente por identificador.

6. Invariantes que deben comprobarse

Después de cada operación de prueba, el equipo deberá verificar:

1. **Relaciones auxiliares consistentes:** si $x.left \neq \varnothing$, entonces $x.left.parent = x$; análogamente para el hijo derecho.
2. **Raíz auxiliar correcta:** `is_aux_root(x)` es verdadera si el padre no reconoce a x como hijo izquierdo ni derecho.
3. **Propagación:** antes de usar o rotar un nodo se aplican las marcas diferidas correspondientes.
4. **Agregados correctos:** `size` y `path_sum` coinciden con el valor local y los subárboles auxiliares.
5. **Sin ciclos auxiliares:** los punteros hijo y padre no forman ciclos.
6. **Bosque válido:** `LINK` no conecta nodos de una misma componente.
7. **Corte válido:** `CUT` solo elimina una arista existente.
8. **Conectividad correcta:** los resultados coinciden con un modelo de referencia basado en BFS o DFS.
9. **Camino correcto:** las consultas agregadas coinciden con la suma obtenida en el modelo de referencia.

Importante. Los Splay Trees auxiliares no se ordenan por el identificador del vértice. El orden auxiliar representa la posición de los vértices sobre un camino.

7. Distribución sugerida de los 120 minutos

Etapa	Actividad	Tiempo
Inicio	Revisión del trabajo preparatorio, interfaces e invariantes	10 min
Resuelto 1	Operaciones splay e instrumentación de rotaciones	18 min
Resuelto 2	Búsqueda, inserción y eliminación programadas	20 min
Resuelto 3	Modelo de bosque de referencia y consultas	17 min
Propuesto 1	Completar y validar el Link-Cut Tree	35 min
Propuesto 2	Consultas, pruebas diferenciales y medición	15 min
Cierre	Verificación individual y organización de evidencias	5 min

8. Ejercicios resueltos

8.1. Ejercicio resuelto 1: ejecutar e instrumentar `splay`

El siguiente programa implementa una rotación general y contabiliza los casos de autoajuste. Se asume que el árbol inicial fue construido respetando el orden BST.

Listing 3: Rotaciones e instrumentación de splay

```python
from dataclasses import dataclass


@dataclass
class SplayStats:
    rotations: int = 0
    zig: int = 0
    zig_zig: int = 0
    zig_zag: int = 0


class Node:
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None
        self.parent = None


class SplayTree:
    def __init__(self):
        self.root = None
        self.stats = SplayStats()

    def _replace_parent_link(self, old, new):
        p = old.parent
        if p is None:
            self.root = new
        elif p.left is old:
            p.left = new
        else:
            p.right = new
        new.parent = p

    def _rotate(self, x):
        p = x.parent
        if p.left is x:              # rotacion derecha
            beta = x.right
            self._replace_parent_link(p, x)
            x.right = p
            p.parent = x
            p.left = beta
        else:                        # rotacion izquierda
            beta = x.left
            self._replace_parent_link(p, x)
            x.left = p
            p.parent = x
            p.right = beta

        if beta is not None:
            beta.parent = p
        self.stats.rotations += 1

    def splay(self, x):
        while x.parent is not None:
            p = x.parent
            g = p.parent

            if g is None:
```

```
                self.stats.zig += 1
                self._rotate(x)
            elif (g.left is p) == (p.left is x):
                self.stats.zig_zig += 1
                self._rotate(p)
                self._rotate(x)
            else:
                self.stats.zig_zag += 1
                self._rotate(x)
                self._rotate(x)
```

Para el árbol con raíz 100, hijo izquierdo 50 y nieto izquierdo 20, ejecutar `splay(20)` produce un caso zig-zig y dos rotaciones. El nuevo nodo raíz es 20.

Qué debe observarse:

- la raíz cambia, pero el recorrido *in-order* se conserva;
- cada rotación cuesta tiempo constante;
- una operación puede realizar varias rotaciones;
- la garantía logarítmica es amortizada, no de peor caso individual.

Visualización

Ademas, implementar la visualización para cada paso del procedimiento splay, es decir se debe dibujar el árbol para cada rotación efectuada, hasta que la raíz sea 20

8.2. Ejercicio resuelto 2: búsqueda, inserción y eliminación

El siguiente código completa las operaciones de diccionario. Cuando una búsqueda falla, se realiza `splay` sobre el último nodo visitado.

Listing 4: Operaciones básicas del Splay Tree

```
class SplayTree(SplayTree):
    def search(self, key):
        x = self.root
        last = None

        while x is not None:
            last = x
            if key == x.key:
                self.splay(x)
                return x
            x = x.left if key < x.key else x.right

        if last is not None:
            self.splay(last)
        return None

    def insert(self, key):
        if self.root is None:
            self.root = Node(key)
            return self.root

        x = self.root
        while True:
            if key == x.key:
```

```
                self.splay(x)
                return x
            if key < x.key:
                if x.left is None:
                    x.left = Node(key)
                    x.left.parent = x
                    self.splay(x.left)
                    return self.root
                x = x.left
            else:
                if x.right is None:
                    x.right = Node(key)
                    x.right.parent = x
                    self.splay(x.right)
                    return self.root
                x = x.right

    def delete(self, key):
        x = self.search(key)
        if x is None or self.root.key != key:
            return False

        left = self.root.left
        right = self.root.right
        if left is not None:
            left.parent = None
        if right is not None:
            right.parent = None

        if left is None:
            self.root = right
            return True

        self.root = left
        maximum = left
        while maximum.right is not None:
            maximum = maximum.right
        self.splay(maximum)
        self.root.right = right
        if right is not None:
            right.parent = self.root
        return True
```

Prueba mínima:

Listing 5: Prueba de las operaciones básicas

```
tree = SplayTree()
for key in [40, 20, 60, 10, 30, 50, 70]:
    tree.insert(key)

assert tree.root.key == 70
assert tree.search(30).key == 30
assert tree.root.key == 30
assert tree.search(25) is None
assert tree.root.key in {20, 30}
assert tree.delete(40)
assert not tree.delete(999)
```

Interpretación. La inserción deja la nueva clave en la raíz. La búsqueda exitosa deja el nodo encontrado en la raíz. La búsqueda fallida aproxima a la raíz el último nodo visitado. La eliminación

separa los subárboles y los reconstruye mediante la idea de `join`.

Visualización

Implementar la visualización de almenos el ultimo árbol resultante.

8.3. Ejercicio resuelto 3: modelo de referencia para el bosque dinámico

Antes de probar un Link-Cut Tree conviene disponer de una implementación sencilla y claramente correcta. El siguiente bosque usa listas de adyacencia. Sus operaciones pueden costar $\Theta(n)$, pero permiten validar la estructura avanzada.

Listing 6: Bosque dinámico de referencia con BFS

```python
from collections import deque


class ReferenceForest:
    def __init__(self, values):
        self.adj = {v: set() for v in values}
        self.value = dict(values)

    def connected(self, source, target):
        if source == target:
            return True
        queue = deque([source])
        seen = {source}
        while queue:
            u = queue.popleft()
            for v in self.adj[u]:
                if v == target:
                    return True
                if v not in seen:
                    seen.add(v)
                    queue.append(v)
        return False

    def link(self, u, v):
        if self.connected(u, v):
            raise ValueError("LINK produciria un ciclo")
        self.adj[u].add(v)
        self.adj[v].add(u)

    def cut(self, u, v):
        if v not in self.adj[u]:
            raise ValueError("La arista no existe")
        self.adj[u].remove(v)
        self.adj[v].remove(u)

    def path(self, source, target):
        queue = deque([source])
        parent = {source: None}
        while queue:
            u = queue.popleft()
            if u == target:
                break
            for v in self.adj[u]:
                if v not in parent:
                    parent[v] = u
                    queue.append(v)
```

```
47
48          if target not in parent:
49              raise ValueError("Los nodos no estan conectados")
50
51          result = []
52          x = target
53          while x is not None:
54              result.append(x)
55              x = parent[x]
56          result.reverse()
57          return result
58
59      def path_sum(self, u, v):
60          return sum(self.value[x] for x in self.path(u, v))
```

Prueba resuelta:

Listing 7: Construcción y consulta del bosque de referencia

```
values = {1: 4, 2: 7, 3: 2, 4: 9, 5: 1, 6: 6, 7: 3}
forest = ReferenceForest(values)

for u, v in [(1, 2), (1, 3), (2, 4),
             (2, 5), (3, 6), (3, 7)]:
    forest.link(u, v)

assert forest.path(4, 6) == [4, 2, 1, 3, 6]
assert forest.path_sum(4, 6) == 28
assert forest.connected(5, 7)

forest.cut(1, 3)
assert not forest.connected(4, 6)
```

Este modelo se reutilizará para las pruebas diferenciales de los ejercicios propuestos.

9. Ejercicios propuestos

9.1. Ejercicio propuesto 1: implementar y validar el Link-Cut Tree

Complete la clase indicada en la sección de especificación. Las operaciones deberán preservar las invariantes del bosque representado y de los Splay Trees auxiliares.

Requisitos funcionales:

1. `is_aux_root(x)` distingue una raíz auxiliar de una conexión virtual;
2. `push(x)` propaga la marca de inversión antes de bajar o rotar;
3. `update(x)` recalcula `size` y `path_sum`;
4. `splay(x)` implementa correctamente zig, zig-zig y zig-zag;
5. `expose(x)` deja el camino raíz–x en un único árbol auxiliar y a x en la raíz auxiliar;
6. `make_root(x)` invierte lógicamente el camino expuesto;
7. `find_root(x)` devuelve la raíz representada vigente;
8. `connected(u,v)` determina si dos nodos pertenecen a la misma componente;

9. `link(u,v)` rechaza una operación que produzca un ciclo;
10. `cut(u,v)` rechaza aristas inexistentes o no directas;
11. `path_sum(u,v)` devuelve la suma de valores del camino simple;
12. un método `validate()` comprueba los invariantes indicados.

Pruebas obligatorias:

1. nodos aislados y consultas de raíz;
2. cadenas, estrellas y árboles balanceados;
3. cambio de raíz mediante `make_root`;
4. varios `expose` consecutivos sobre nodos distintos;
5. enlaces válidos e intento de crear un ciclo;
6. cortes de aristas cercanas a la raíz y de aristas internas;
7. intento de cortar una arista inexistente;
8. consulta de suma con valores positivos, cero y negativos;
9. secuencias que alternan `link`, `cut`, `make_root` y consultas;
10. ejecución de `validate()` después de cada operación.

El equipo deberá instrumentar contadores de rotaciones, operaciones splay, exposiciones, inversiones perezosas y actualizaciones de agregados.

Visualización obligatoria de `EXPOSE`

Además de implementar la operación, el equipo deberá generar en Python una visualización de la ejecución de `EXPOSE(v)`. La visualización deberá incluir, como mínimo:

1. el árbol representado antes de ejecutar la operación;
2. la descomposición inicial en caminos preferidos;
3. los Splay Trees auxiliares antes de la operación;
4. cada operación `splay` y cada cambio de arista preferida realizado durante `EXPOSE(v)`;
5. el Splay Tree auxiliar resultante que representa el camino desde la raíz representada hasta v;
6. el árbol representado después de la operación, mostrando que sus aristas no fueron modificadas.

Cada imagen deberá identificar el número de paso, el nodo procesado y la operación realizada.

9.2. Ejercicio propuesto 2: pruebas diferenciales y análisis experimental

Compare la implementación del Link-Cut Tree con el bosque de referencia del ejercicio resuelto 3. Ambas estructuras deberán recibir exactamente la misma secuencia válida de operaciones.

Escenario A: corrección determinista

Use los valores:

v	1	2	3	4	5	6	7
`value`(v)	4	7	2	9	1	6	3

y construya el árbol con aristas $(1,2)$, $(1,3)$, $(2,4)$, $(2,5)$, $(3,6)$ y $(3,7)$. Compruebe:

$$\texttt{path_sum(4,5)} = 17,$$
$$\texttt{path_sum(4,6)} = 28,$$
$$\texttt{path_sum(6,7)} = 11.$$

Después ejecute cortes y enlaces válidos, verificando conectividad y sumas tras cada modificación.

Escenario B: pruebas aleatorias diferenciales

Para $n \in \{30, 60, 120\}$:

1. cree n vértices aislados con valores reproducibles;
2. genere al menos 5000 operaciones válidas con una semilla fija;
3. combine `link`, `cut`, `connected` y `path_sum`;
4. ante una discrepancia, guarde la semilla y toda la secuencia ejecutada;
5. informe cuántas operaciones de cada tipo fueron verificadas.

Escenario C: comportamiento experimental

Para $n \in \{100, 200, 400, 800, 1600\}$, construya:

1. una cadena;
2. una estrella;
3. un árbol generado aleatoriamente.

Ejecute lotes de consultas de conectividad y suma de camino, junto con secuencias mixtas de enlace y corte. Registre tiempo, rotaciones, operaciones splay y exposiciones.

Hipótesis experimental

No se debe intentar demostrar la cota amortizada únicamente con tiempos. La conclusión debe distinguir peor caso individual, costo amortizado, constantes de implementación y forma de las instancias.

10. Resultados mínimos requeridos

Los resultados mínimos son:

1. tabla de casos deterministas y resultados esperados/obtenidos;
2. cantidad de operaciones diferenciales verificadas por tipo;
3. tiempo total frente a n para cadena, estrella y árbol aleatorio;
4. rotaciones promedio por operación frente a n;
5. exposiciones y operaciones splay por lote de consultas;
6. evidencia de rechazo de enlaces cíclicos y cortes inválidos;
7. una secuencia reproducible que combine enlace, corte, cambio de raíz y consulta de camino.

11. Preguntas de análisis

Responda utilizando evidencia de la implementación y los experimentos:

1. ¿Qué información pertenece al árbol representado y cuál a los árboles auxiliares?
2. ¿Por qué una rotación splay no modifica las aristas reales del bosque?
3. ¿Qué resultado estructural deja `EXPOSE(v)`?
4. ¿Por qué `MAKE-ROOT(v)` requiere invertir el camino expuesto?
5. ¿Qué errores aparecen si no se propaga `reversed` antes de rotar?
6. ¿Cómo evita `LINK` la formación de ciclos?
7. ¿Cómo comprueba `CUT` que dos nodos forman una arista directa?
8. ¿Por qué `PATH-SUM(u,v)` se resuelve después de cambiar la raíz a u y exponer v?
9. ¿Qué patrones de entrada generaron más rotaciones por operación?
10. ¿Por qué una operación individual puede costar $\Theta(n)$ y aun así la estructura ofrecer $\mathcal{O}(\log n)$ amortizado?
11. ¿Qué errores de implementación detectó el modelo BFS/DFS?
12. ¿Qué aspectos de esta implementación académica limitarían su uso directo en un sistema real?

12. Producto de entrega

Producto de entrega

El equipo entregará:

1. código fuente completo y organizado, tanto ejercicios resueltos como propuestos;
2. pruebas automatizadas de operaciones e invariantes;
3. generador reproducible de secuencias válidas de operaciones;
4. archivo CSV con mediciones y contadores;
5. tablas y gráficos solicitados;
6. informe PDF con todo lo previo y metodología, resultados y respuestas de análisis;
7. conclusiones grupales de 250 a 350 palabras.

13. Tabla de evaluación

Criterio	Indicador de logro	Puntaje
Asistencia y participación en el laboratorio	Asiste, permanece durante la práctica y participa en implementación, pruebas, análisis y revisiones. Se determina individualmente.	4
Corrección y funcionamiento	Los ejercicios resueltos, el Link-Cut Tree, el modelo de referencia, las pruebas y los verificadores funcionan; las operaciones preservan invariantes y ambas estructuras producen resultados equivalentes.	8
Análisis de complejidad y resultados	Relaciona rotaciones, exposiciones, peor caso y costo amortizado; interpreta los experimentos sin confundir complejidad asintótica con rendimiento observado.	3
Dominio individual	Explica Splay Trees, árbol representado, árboles auxiliares, `EXPOSE`, inversión, `LINK`, `CUT` y consulta de caminos; ejecuta o modifica una prueba durante la revisión.	4
Total		**19**

Referencias

[1] Sleator, D. D. y Tarjan, R. E. (1985). *Self-adjusting binary search trees*. Journal of the ACM, 32(3).

[2] Sleator, D. D. y Tarjan, R. E. (1983). *A data structure for dynamic trees*. Journal of Computer and System Sciences, 26(3), 362–391.

[3] Cormen, T. H., Leiserson, C. E., Rivest, R. L. y Stein, C. (2022). *Introduction to Algorithms* (4.ª ed.). MIT Press.

[4] MIT OpenCourseWare. *Dynamic graphs and Link-Cut Trees*. https://ocw.mit.edu/